



UNIVERSIDAD POLITÉCNICA DE JUVENTINO ROSAS

PROYECTO INTEGRADOR

Digitalización de reportes de incidentes para bomberos

Presenta:

Gasca González Juan Pablo
Lara Ariza Abraham Isaí
Olivares Miranda Merari Jocelyn

Nombre del equipo:

'404ERROR'

Docente:

Mtro. Miguel Ángel Arreguín Juárez

Santa Cruz de Juventino Rosas, Gto. 30 de noviembre de 2024.

Contenido

Contenido	I
Resumen	III
Abstract	IV
Lista de figuras	V
Lista de tablas	VII
1 Introducción	1
1.1. Planteamiento del problema	1
1.2. Objetivo general	1
1.3. Objetivos particulares	1
1.4. Antecedentes	2
1.4.1. Diseño e implementación de una aplicación móvil para la gestión de emergencias	2
1.4.2. Sistemas de automatización de emergencias: Un enfoque digital	3
1.4.3. Automatización de la gestión de reportes de emergencias en ser- vicios públicos	5
1.4.4. Desarrollo de una aplicación para la gestión digital de emergencias	6
1.4.5. Desarrollo de una aplicación para la gestión de emergencias en tiempo real	8
1.5. Alcances del trabajo	9
1.5.1. Capítulo 1. Introducción	9
1.5.2. Capítulo 2. Marco teórico	9
1.5.3. Capítulo 3. Marco metodológico	10
1.5.4. Capítulo 4. Pruebas y resultados	10
1.5.5. Capítulo 5. Conclusiones	10
1.6. Cronograma de actividades	10

2	Marco teórico	12
2.1.	Flutter: Marco de desarrollo	12
2.1.1.	Definición y arquitectura de Flutter	12
2.1.2.	Desarrollo de interfaces de usuario en Flutter	13
2.1.3.	Ecosistema de Flutter	17
2.2.	Lenguaje de programación Dart	17
2.2.1.	Sintaxis de Dart	18
2.2.2.	Características principales de Dart	22
2.3.	Visual Studio Code: Entorno de desarrollo	22
2.3.1.	Git y GitHub para el control de versiones	23
2.4.	Arquitectura de aplicaciones móviles	24
2.5.	Bases de datos y Firebase	27
3	Marco metodológico	29
3.1.	Enfoque metodológico	29
3.2.	Requerimientos del usuario	30
3.3.	Desarrollo de la aplicación	31
3.3.1.	Diseño de la interfaz	31
3.3.2.	Estructura general de archivos y directorios	33
3.4.	Técnicas de validación	38
3.5.	Documentación	39
4	Pruebas y resultados	40
5	Conclusiones	44
5.1.	Trabajo a futuro	44
	Bibliografía	46
A	Reporte de incidentes de bomberos	48

Resumen

El presente proyecto busca optimizar el proceso de registro, almacenamiento y consulta de reportes generados por el cuerpo de bomberos. En su estado actual, este proceso es manual, consumiendo un tiempo significativo debido al llenado físico de formularios y su posterior carga a la nube.

La solución propuesta es una aplicación móvil desarrollada en Flutter, utilizando Firebase como base de datos central. Esta herramienta les permitirá capturar datos de manera digital, reducir el tiempo de elaboración de los reportes, almacenarlos de forma segura, y consultarlos o modificarlos según sea necesario.

Abstract

This project seeks to optimize the process of registration, storage and consultation of reports generated by the fire department. In its current state, this process is manual, consuming significant time due to the physical filling out of forms and their subsequent uploading to the cloud.

The proposed solution is a mobile application developed in Flutter, using Firebase as the central database. This tool will allow them to capture data digitally, reduce report preparation time, store it securely, and consult or modify it as necessary.

Lista de figuras

1.1. Pantalla principal del sistema de asistencia a emergencias 9-1-1 por la Universidad Nacional Pedro Henríquez Ureña (UNPHU).	2
1.2. Pantalla menú del sistema de asistencia a emergencias 9-1-1 por la Universidad Nacional Pedro Henríquez Ureña (UNPHU).	3
1.3. Pantalla reportes del sistema de asistencia a emergencias 9-1-1 por la Universidad Nacional Pedro Henríquez Ureña (UNPHU).	3
1.4. Pantalla inicial del sistema de agencia metropolitana de tránsito.	4
1.5. Pantalla menú, reporte y localización del sistema de agencia metropolitana de tránsito.	4
1.6. Pantalla envío de alertas del sistema de servicios de emergencia.	5
1.7. Pantalla alerta a bomberos del sistema de servicios de emergencia.	5
1.8. Pantalla inicial del sistema de gestión digital de emergencias atendidas por bomberos.	6
1.9. Funciones del sistema de gestión digital de emergencias atendidas por bomberos.	6
1.10. Pantalla reporte del sistema de gestión digital de emergencias atendidas por bomberos.	7
1.11. Maqueta de interfaz de aplicación para la gestión de emergencias.	8
1.12. Maqueta de interfaz 1 de aplicación para la gestión de emergencias.	9
1.13. Cronograma de actividades a desarrollar para el proyecto integrador.	11
2.1. Logo de Flutter.	12
2.2. Ejemplo de árbol de <i>widgets</i> en Flutter.	13
2.3. Diagrama de integración de Firebase.	17
2.4. Logo de Dart.	17
2.5. Sintaxis de Dart.	18
2.6. Sintaxis de cadenas en Dart.	18
2.7. Sintaxis de números en Dart.	18
2.8. Sintaxis de booleanos en Dart.	19
2.9. Sintaxis de listas en Dart.	19

2.10. Sintaxis de mapas en Dart.	19
2.11. Sintaxis de datos dinámicos en Dart.	19
2.12. Sintaxis de condicional <i>if else</i> en Dart.	20
2.13. Sintaxis de condicional <i>switch</i> en Dart.	20
2.14. Sintaxis de ciclo <i>for</i> en Dart.	20
2.15. Sintaxis de clases y objetos en Dart.	21
2.16. Sintaxis de IU en Dart.	21
2.17. Logo de Visual Studio Code.	23
2.18. Logo de Git y GitHub.	24
2.19. Diagrama del patrón MVC.	25
2.20. Diagrama del patrón MVVM.	26
2.21. Diagrama del patrón BloC.	26
2.22. Logo de Firebase.	27
3.1. Diagrama de metodología ágil.	30
3.2. Diagrama de funcionamiento de la aplicación.	32
3.3. Paleta de colores seleccionado para la aplicación.	32
3.4. Colores finales seleccionados para la aplicación.	33
3.5. Carpetas principales del proyecto.	34
3.6. Código de <i>main.dart</i>	34
3.7. Código muestra del ícono y mensaje de bienvenida.	35
3.8. Código muestra de <i>ListTile</i>	36
3.9. Código muestra de <i>AppBar</i>	36
3.10. Código muestra de <i>Floating Action Button</i>	37
3.11. Código muestra de <i>my_button.dart</i>	37
3.12. Código muestra de <i>my_textfield.dart</i>	38
3.13. Código muestra de <i>theme.dart</i>	38
4.1. Pantalla de <i>Login</i> de la aplicación.	41
4.2. Pantalla de reportes de la aplicación.	42
4.3. Pantalla del <i>dashboard</i> de la aplicación.	43

Lista de tablas

2.1. Diferencias entre tipos de <i>widgets</i>	13
2.2. Tipos comunes de <i>widgets</i> en Flutter.	16
2.3. Características de Git y GitHub.	24

Capítulo 1

Introducción

1.1. Planteamiento del problema

El problema específico que se aborda con la aplicación móvil es la ineficiencia en la documentación de incidentes atendidos por los cuerpos de bomberos. En la actualidad, estos reportes se generan manualmente, lo que da lugar a errores en la captura de datos, retrasos en la documentación y dificultades en la gestión de la información. Esta falta de automatización no solo afecta la eficiencia operativa del cuerpo de bomberos, sino que también puede comprometer la calidad de la información. Por lo tanto, es fundamental desarrollar una aplicación que digitalice el proceso de llenado y almacenamiento de reportes, permitiendo una captura de datos más rápida y precisa, lo que contribuirá a una mejor gestión de emergencias y a la organización interna de los cuerpos de bomberos.

1.2. Objetivo general

Desarrollar una aplicación móvil destinada a la creación, manejo y consulta de reportes de incidentes atendidos por bomberos, la cual permita digitalizar el proceso de llenado y almacenamiento de estos.

1.3. Objetivos particulares

1. Crear una interfaz de usuario para la aplicación móvil, que sea intuitiva y didáctica para llenar por parte del cuerpo de bomberos.
2. Implementar validaciones de datos en la aplicación para verificar la coherencia de la información ingresada en los campos.

3. Implementar un sistema de creación de documentos digitalizados en formato oficial de bomberos que pase los datos de la aplicación al documento para la impresión y almacenamiento del mismo.
4. Crear una base de datos para guardar los reportes y administrarlos de tal manera que sea confiable la información.
5. Realizar pruebas de uso con los bomberos para recoger sus opiniones y sugerencias, reajustando la aplicación según estas.

1.4. Antecedentes

1.4.1. Diseño e implementación de una aplicación móvil para la gestión de emergencias

Este proyecto, desarrollado en la Universidad Nacional Pedro Henríquez Ureña (UNPHU), aborda la creación de una aplicación móvil orientada a mejorar la gestión de emergencias en tiempo real [1]. La aplicación permite automatizar la recolección de datos de incidentes y su almacenamiento en una base de datos centralizada. Se hace hincapié en la importancia de reducir los errores humanos en la documentación de emergencias y en la capacidad de la aplicación para generar reportes oficiales de manera rápida y precisa. Además, el proyecto estudia la usabilidad de la interfaz para garantizar que el personal de emergencias pueda utilizarla eficientemente.

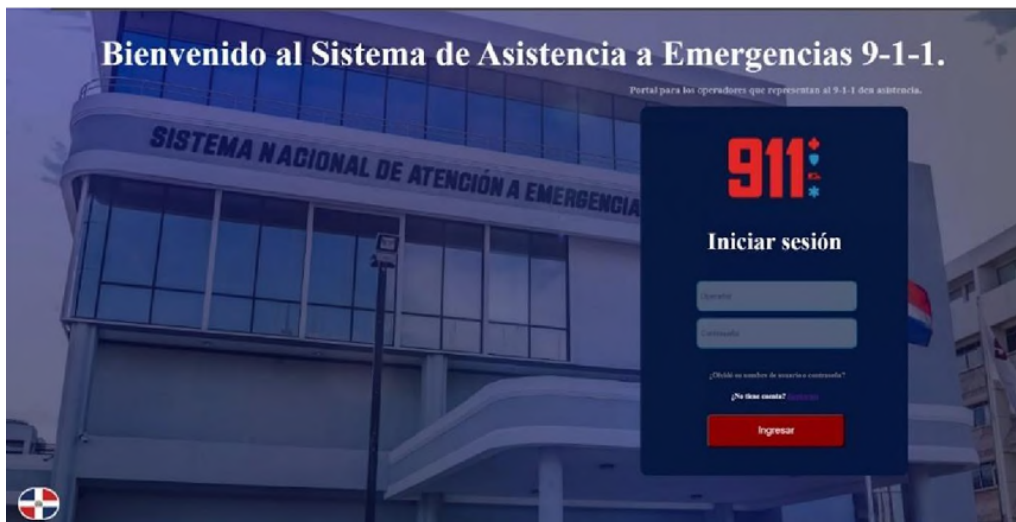


Figura 1.1: Pantalla principal del sistema de asistencia a emergencias 9-1-1 por la Universidad Nacional Pedro Henríquez Ureña (UNPHU).



Figura 1.2: Pantalla menú del sistema de asistencia a emergencias 9-1-1 por la Universidad Nacional Pedro Henríquez Ureña (UNPHU).

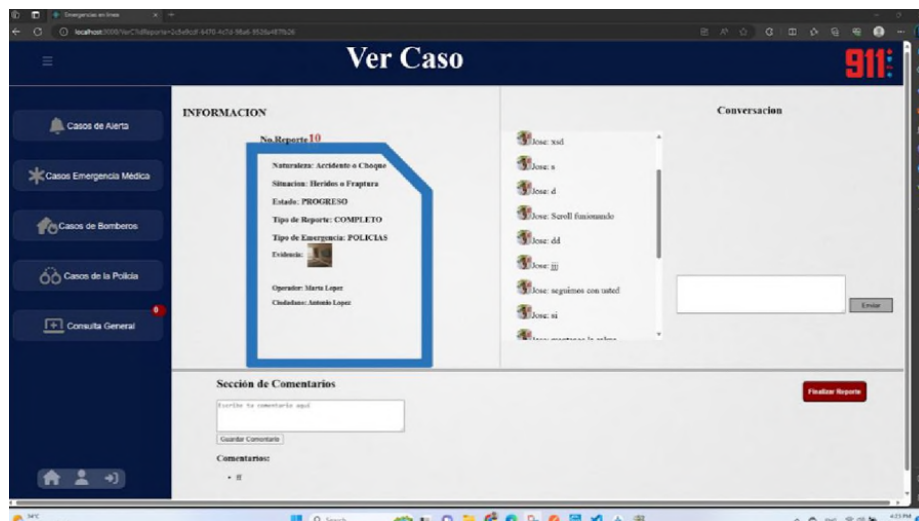


Figura 1.3: Pantalla reportes del sistema de asistencia a emergencias 9-1-1 por la Universidad Nacional Pedro Henríquez Ureña (UNPHU).

1.4.2. Sistemas de automatización de emergencias: Un enfoque digital

Este artículo se enfoca en la integración de tecnologías digitales para la automatización de la gestión de emergencias. Publicado en *ProQuest Dissertations Theses Global*, examina cómo las herramientas tecnológicas han mejorado la eficiencia operativa en la captura de datos y la generación de reportes durante situaciones críticas. El estudio también analiza varios casos de éxito de la implementación de estas tecnologías en orga-

nizaciones de respuesta a emergencias, subrayando la reducción de tiempos de respuesta y el incremento en la precisión de los reportes [2].



Figura 1.4: Pantalla inicial del sistema de agencia metropolitana de tránsito.

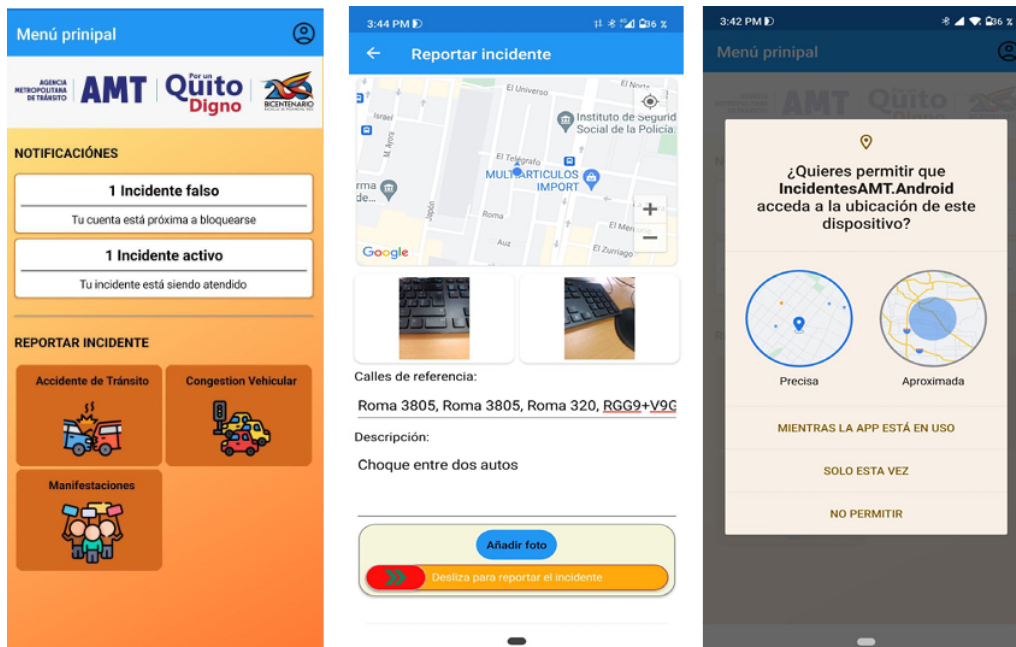


Figura 1.5: Pantalla menú, reporte y localización del sistema de agencia metropolitana de tránsito.

1.4.3. Automatización de la gestión de reportes de emergencias en servicios públicos

Este trabajo, realizado en la Universidad de Córdoba, presenta una solución basada en una aplicación móvil diseñada para los servicios públicos de emergencia [3]. La aplicación permite gestionar de manera automática los reportes de incidentes, con un enfoque en la integración con bases de datos gubernamentales. El estudio incluye pruebas de campo realizadas con diferentes cuerpos de emergencia, como bomberos y servicios de ambulancia, que demostraron la capacidad de la herramienta para reducir la carga administrativa y mejorar la precisión de los reportes.

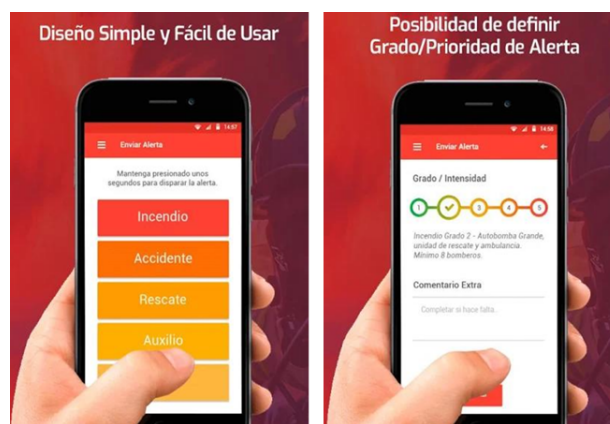


Figura 1.6: Pantalla envío de alertas del sistema de servicios de emergencia.

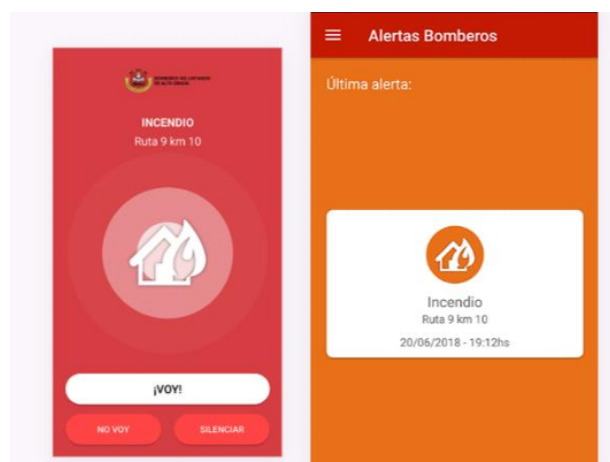


Figura 1.7: Pantalla alerta a bomberos del sistema de servicios de emergencia.

1.4.4. Desarrollo de una aplicación para la gestión digital de emergencias

Este proyecto de la Universidad Estatal Península de Santa Elena (UPSE) se centra en la creación de una aplicación móvil que permite gestionar digitalmente las emergencias atendidas por los bomberos [4]. La investigación destaca la importancia de contar con una base de datos segura para el almacenamiento de los reportes y la capacidad de la aplicación para generar documentos en formato oficial de bomberos. También se evalúa el impacto de esta herramienta en la reducción de errores en la captura de datos y el aumento de la eficiencia en la generación de reportes.

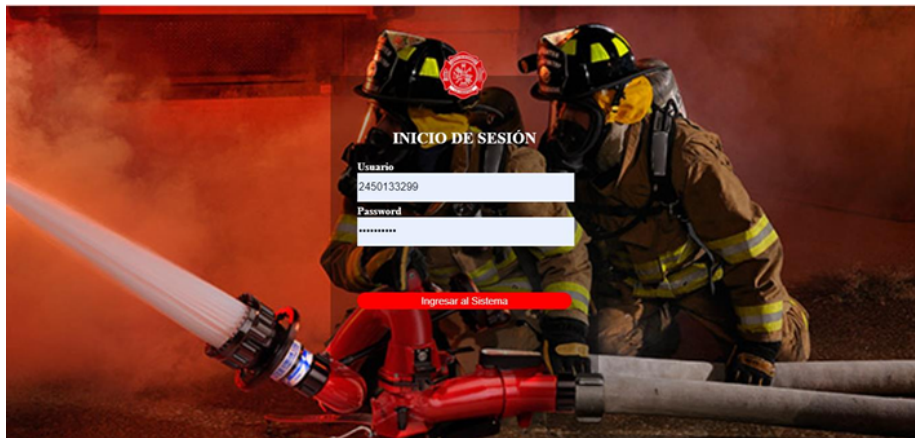


Figura 1.8: Pantalla inicial del sistema de gestión digital de emergencias atendidas por bomberos.

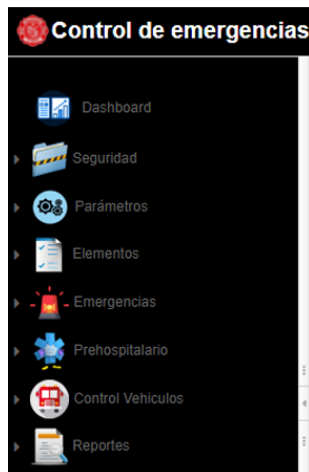


Figura 1.9: Funciones del sistema de gestión digital de emergencias atendidas por bomberos.

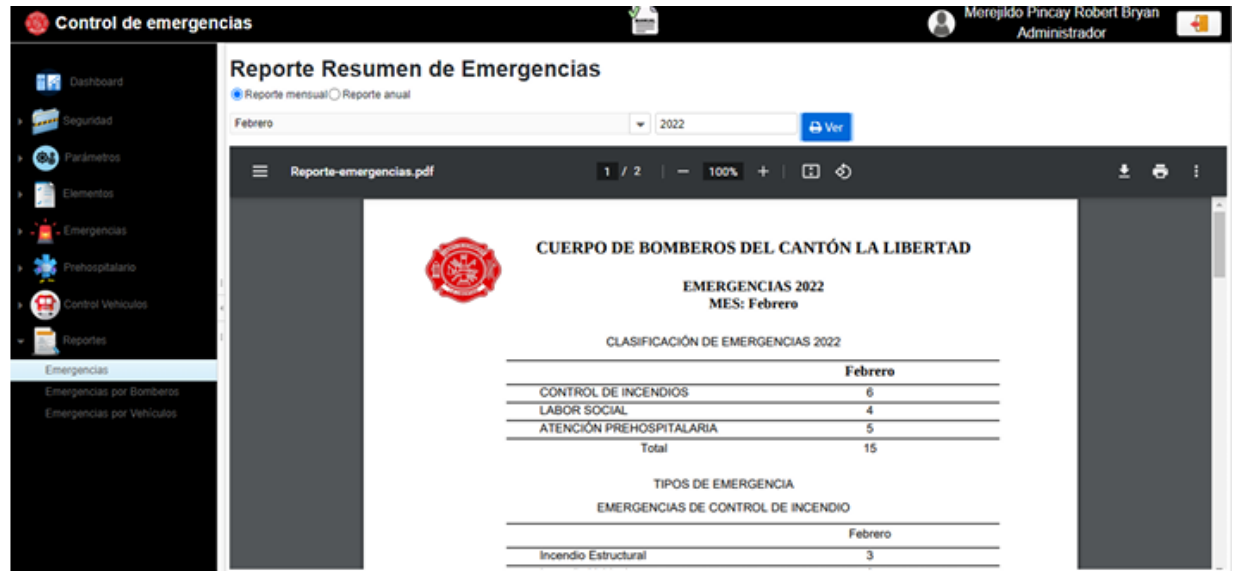


Figura 1.10: Pantalla reporte del sistema de gestión digital de emergencias atendidas por bomberos.

1.4.5. Desarrollo de una aplicación para la gestión de emergencias en tiempo real

Este trabajo de fin de grado realizado por Rubén Puga Roldán en la Universidad de Málaga se centra en la creación de una aplicación móvil que facilita la gestión y documentación de emergencias. La investigación aborda los desafíos técnicos de la integración de tecnologías móviles en la captura y automatización de reportes de incidentes, con un enfoque especial en la interfaz de usuario y la precisión en la recolección de datos [5]. Además, el proyecto evalúa el impacto de la aplicación en la mejora de la eficiencia operativa de los cuerpos de bomberos.

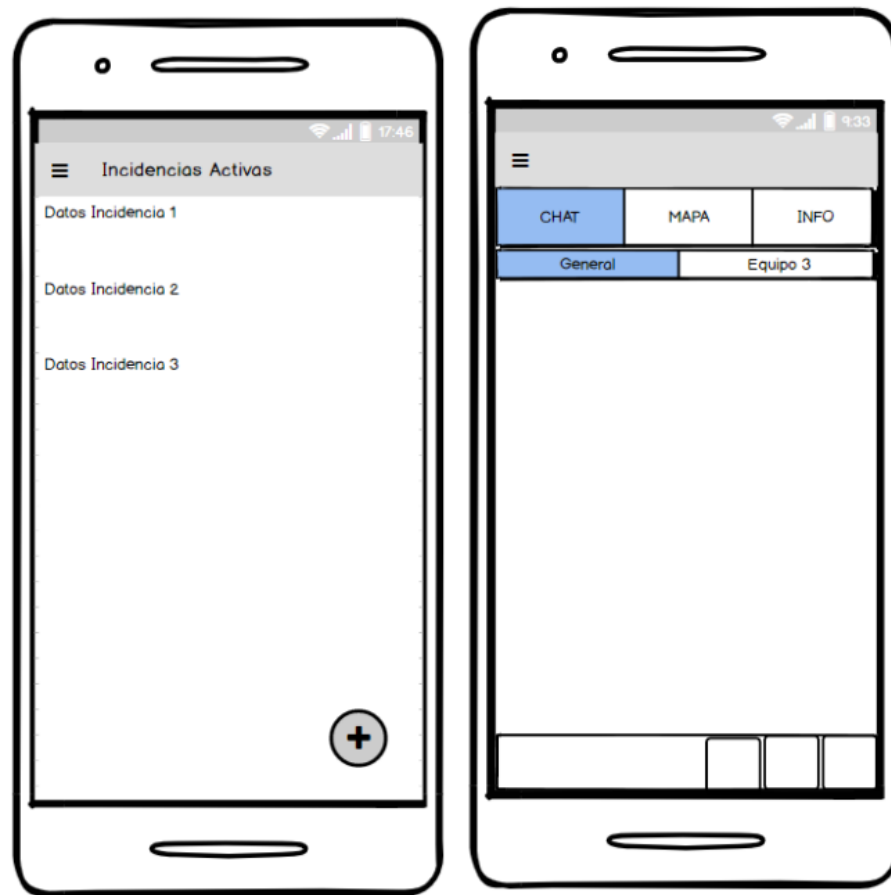


Figura 1.11: Maqueta de interfaz de aplicación para la gestión de emergencias.

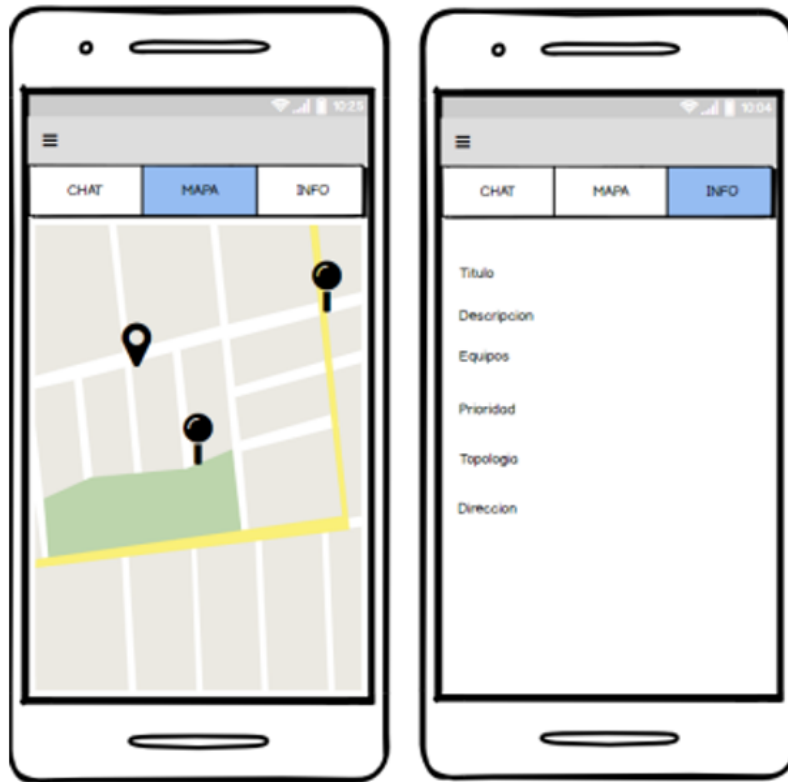


Figura 1.12: Maqueta de interfaz 1 de aplicación para la gestión de emergencias.

1.5. Alcances del trabajo

1.5.1. Capítulo 1. Introducción

En este primer capítulo se establecieron las bases para los capítulos subsiguientes y se justificó la importancia de desarrollar una aplicación móvil. La aplicación digitalizará el llenado y la gestión de reportes de incidentes para bomberos. Se comenzó con la presentación del problema central, que radica en mejorar la eficiencia y precisión del proceso de reporte de incidentes. Posteriormente, se formularon los objetivos y se revisaron antecedentes relevantes relacionados con la gestión de incidentes y el uso de nuevas tecnologías móviles en emergencias.

1.5.2. Capítulo 2. Marco teórico

En este capítulo se proporcionará una base sólida de conocimientos que respalde el desarrollo y la comprensión de la aplicación móvil. Este capítulo incluirá:

- Una introducción a Flutter y al lenguaje Dart, con énfasis en las herramientas que facilitan el desarrollo de interfaces gráficas interactivas y modernas.

- La generación digital de reportes en formatos oficiales, con la capacidad de imprimirlos, almacenarlos, etc.

1.5.3. Capítulo 3. Marco metodológico

Este capítulo detalla la metodología y el enfoque utilizado para el diseño, desarrollo e implementación de la aplicación móvil. Incluirá:

- Descripción del diseño de la aplicación, destacando el uso de Flutter y Dart para el desarrollo de una solución modular y eficiente.
- Elaboración del diagrama de flujo que explique el funcionamiento interno de la aplicación, desde la validación de datos hasta la creación de reportes.

1.5.4. Capítulo 4. Pruebas y resultados

Este capítulo presenta los resultados obtenidos durante el desarrollo del prototipo de la aplicación. Incluirá:

- Evaluación del flujo del prototipo, asegurando que sea fluida y funcione correctamente de forma inicial.

1.5.5. Capítulo 5. Conclusiones

En este capítulo se presentarán las conclusiones finales del desarrollo de la aplicación. Incluirá:

- Un resumen de los logros clave, como el aprendizaje básico desde cero del lenguaje de programación Dart.
- Recomendaciones para futuros desarrollos, como validaciones de formularios y creación de los formularios para la digitalización de los reportes.

1.6. Cronograma de actividades

A continuación se adjunta el cronograma de actividades que se ira desarrollando durante este cuatrimestre en curso:

Inicia: 9 Septiembre, termina: 19 de Diciembre								Total de semanas 15				Días hábiles								Días inhábiles			
Cronograma de actividades para proyecto integrador: Automatización digital de reportes de incidentes para bomberos																							
Periodo		Septiembre				Octubre				Noviembre				Diciembre									
Actividades a realizar	Semana 1	Semana 2	Semana 3	Semana 4	Semana 5	Semana 6	Semana 7	Semana 8	Semana 9	Semana 10	Semana 11	Semana 12	Semana 13	Semana 14	Semana 15								
	L,M,M,J,V	L,M,M,J,V	L,M,M,J,V	L,M,M,J,V	L,M,M,J,V	L,M,M,J,V	L,M,M,J,V	L,M,M,J,V	L,M,M,J,V	L,M,M,J,V	L,M,M,J,V	L,M,M,J,V	L,M,M,J,V	L,M,M,J,V	L,M,M,J,V								
Inscripción a curso de Flutter/Dart																							
Introducción a Dart (variables, tipos de datos)																							
Control de flujo y funciones en Dart																							
Desarrollo de interfaces básicas en Flutter presentación y entrega de avances del proyecto																							

Figura 1.13: Cronograma de actividades a desarrollar para el proyecto integrador.

Anexos

Apéndice A. Formato de reporte de incidentes del cuerpo de bomberos.

Capítulo 2

Marco teórico

2.1. Flutter: Marco de desarrollo

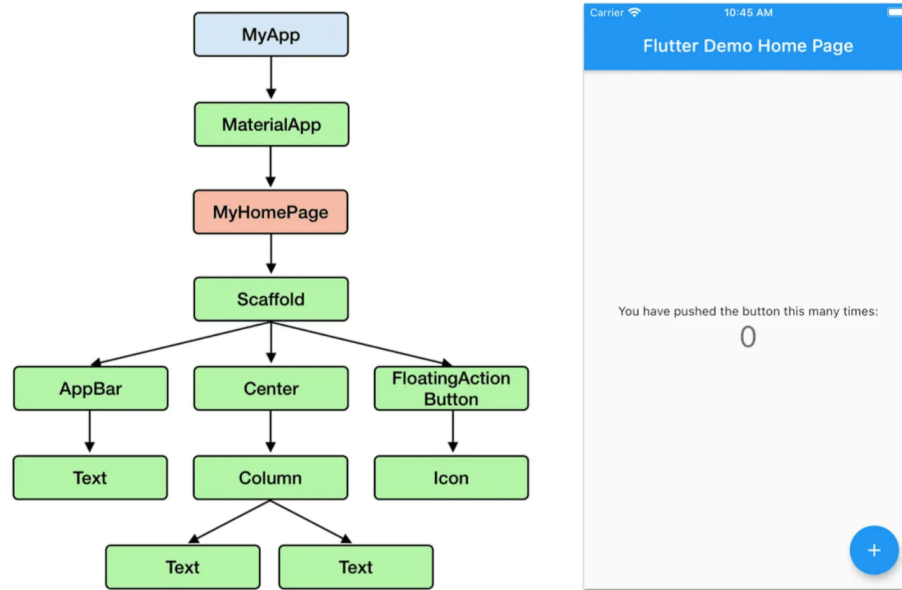
2.1.1. Definición y arquitectura de Flutter

Flutter es un kit de desarrollo móvil o SDK (*Software Development Kit*) de código abierto desarrollado por Google y que facilita la creación de aplicaciones móviles multi-plataformas [6]. Con Flutter se pueden crear todo tipo de aplicaciones para dispositivos móviles de forma rápida y sencilla, tanto para dispositivos Android o iOS, como *web apps* (aplicaciones web).



Figura 2.1: Logo de Flutter.

Flutter controla cada píxel que se dibuja en la pantalla, su *framework* ofrece un amplio conjunto de componentes de UI (*User interface*, llamados *widgets*) que se asemejan mucho a los controles de UI nativos en iOS y Android. Los *widgets* pequeños y de un solo propósito se componen juntos para crear otros más complejos y especializados que representan la interfaz de usuario [6]. Por lo tanto, toda la aplicación está representada por un árbol de *widgets*, por ejemplo:

Figura 2.2: Ejemplo de árbol de *widgets* en Flutter.

2.1.2. Desarrollo de interfaces de usuario en Flutter

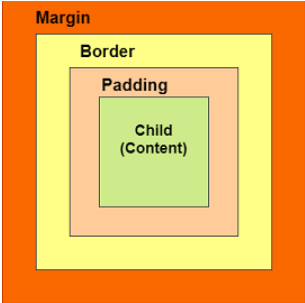
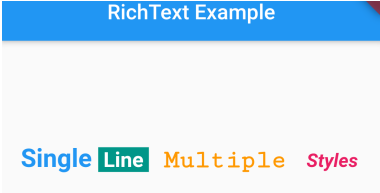
El desarrollo de interfaces de usuario (UI) es un componente crucial en cualquier aplicación móvil, ya que determina cómo los usuarios interactúan con la aplicación y la calidad de su experiencia. Flutter destaca por su flexibilidad y facilidad de uso, lo que permite crear interfaces visualmente atractivas y altamente personalizables.

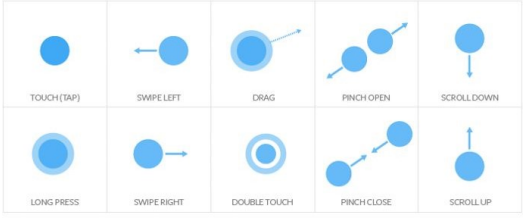
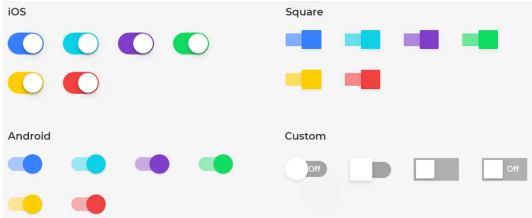
Además, Flutter permite la implementación de diseños personalizados gracias a su arquitectura de *widgets* altamente componibles, lo que facilita la creación de interfaces con efectos visuales avanzados, animaciones fluidas y transiciones interactivas [7]. Estos se dividen en dos grandes categorías:

<i>Stateless Widgets</i>	<i>Stateful Widgets</i>
No pueden cambiar de estado una vez creados.	Pueden actualizar su contenido a lo largo del ciclo de vida de la aplicación.
Se usa para componentes estáticos como íconos o textos.	Se usa para elementos dinámicos que respondan a interacciones del usuario, como formularios o animaciones.

Tabla 2.1: Diferencias entre tipos de *widgets*.

Algunos de los *widgets* más comunes se muestran en la siguiente tabla:

Tipos de <i>widgets</i>	Ejemplos
Diseño	Container: Contenedor rectangular para organizar y personalizar otros <i>widgets</i> dentro de él. Permite agregar márgenes, rellenos, bordes y aplicar decoraciones como sombras y esquinas redondeadas.  Text: Para mostrar texto. Es altamente configurable y admite múltiples estilos de texto como tamaño, color, alineación y decoración.  Image: Para mostrar imágenes ya sea desde la red, un archivo local o un recurso.  Icon: Representaciones gráficas de acciones comunes, como una lupa para búsqueda o una papelera para eliminar. 

Tipos de <i>widgets</i>	Ejemplos
Interacción	■ <i>GestureDetector</i>: Detecta diferentes tipos de gestos del usuario, como toques, deslizamientos y más.
	
	■ <i>Button</i>: Para manejar pulsaciones. 
	■ <i>TextField</i>: Para que el usuario ingrese datos. 
	■ <i>Slider</i>: Para seleccionar un valor en un rango.  ■ <i>Switch</i>: Para conmutar entre dos estados (encendido/apagado). 

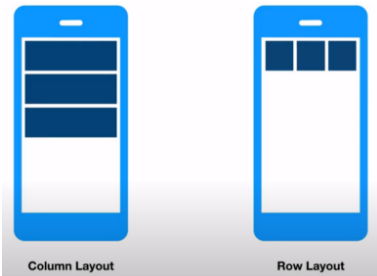
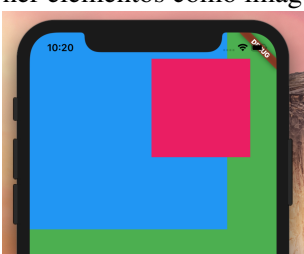
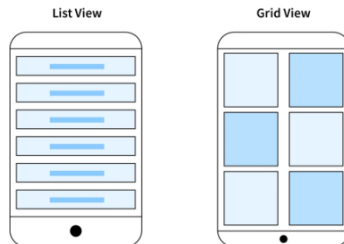
Tipos de <i>widgets</i>	Ejemplos
Disposición	Column y Row: Organizan <i>widgets</i> en líneas verticales u horizontales, utilizando propiedades como <i>mainAxisAlignment</i> y <i>crossAxisAlignment</i> para controlar el espaciado y la alineación.  Stack: Superpone <i>widgets</i> unos sobre otros, ideal para diseños donde se necesita superponer elementos como imágenes y textos.  ListView y GridView: Muestran listas de elementos de manera vertical o en cuadrículas. Ambos admiten un desplazamiento fluido y pueden manejar grandes cantidades de datos. 

Tabla 2.2: Tipos comunes de *widgets* en Flutter.

2.1.3. Ecosistema de Flutter

Su ecosistema es rico en herramientas y bibliotecas que facilitan el desarrollo, prueba, y mantenimiento de aplicaciones. Cuenta con un potente conjunto de herramientas, como “Flutter DevTools”, que permiten realizar análisis de rendimiento, depuración visual, y gestión de estado en tiempo real. También integra de manera nativa con “Firebase”, lo que permite utilizar funcionalidades como bases de datos en tiempo real, autenticación, analítica, y almacenamiento en la nube, lo que es ideal para aplicaciones que necesitan un *backend* potente y escalable [8].

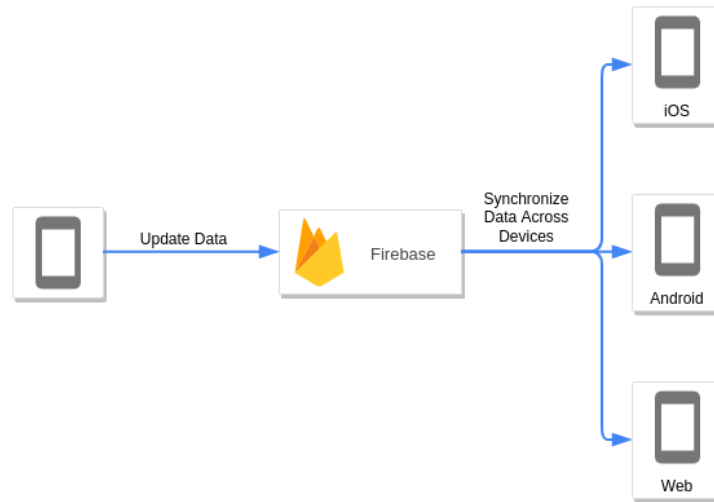


Figura 2.3: Diagrama de integración de Firebase.

La comunidad de Flutter está en crecimiento, y Google ofrece un soporte continuo con actualizaciones regulares.

2.2. Lenguaje de programación Dart

Dart es un lenguaje *open source* desarrollado en Google lanzado por primera vez en 2011, creado con la visión de ofrecer una alternativa más estructurada y eficiente para el desarrollo web en comparación con JavaScript [9]. Ofrece una sintaxis clara y concisa, lo que facilita su aprendizaje y legibilidad, fue concebido originalmente con un enfoque en el desarrollo web, Dart ha demostrado ser un lenguaje versátil, capaz de abordar problemas en múltiples plataformas, desde aplicaciones móviles hasta aplicaciones de escritorio.



Figura 2.4: Logo de Dart.

Un rasgo distintivo de Dart es su máquina virtual (VM), que permite una ejecución rápida y un desarrollo más fluido, gracias a características como el “Hot Reload”, que facilita la visualización instantánea de cambios en el código sin reiniciar toda la aplicación [9].

2.2.1. Sintaxis de Dart

Dart admite conceptos de programación como interfaces y clases. Las colecciones de Dart se pueden usar para replicar estructuras de datos como matrices, genéricos y tipo opcional [10]. La siguiente sintaxis muestra un programa escrito en Dart:

```
1. void main() {  
2.     print("Hola mundo !");  
3. }
```

Figura 2.5: Sintaxis de Dart.

■ Tipos de datos:

1. Cadenas. Representa una secuencia de caracteres, los valores se especifican en comillas simples o dobles.

```
1. void main() {  
2.     String cadena1 = 'Esto es una cadena';  
3.     String cadena2 = "Esto es otra cadena";  
4.     print("Cadena 1: " + cadena1 + " Cadena 2: " + cadena2) ;  
5. }
```

Figura 2.6: Sintaxis de cadenas en Dart.

2. Números. Representan literales numéricos entero y doble.

```
1. void main() {  
2.     int entero = 32;  
3.     double decimal = 32.5 ;  
4.     print("Entero: " + entero.toString() + " Decimal: " + decimal.toString()) ;  
5. }
```

Figura 2.7: Sintaxis de números en Dart.

3. Booleanos. Utiliza la palabra clave *bool* para representar valores true o false.

```
1. void main() {  
2.     bool bandera = true ;  
3.     print(bandera);  
4. }
```

Figura 2.8: Sintaxis de booleanos en Dart.

4. Listas. Representan una colección de objetos.

```
1. void main() {  
2.     var lista = [1,2,3,4,5];  
3.     print(lista);  
4. }
```

Figura 2.9: Sintaxis de listas en Dart.

5. Mapas. Representan una colección de objetos como llave (valor).

```
1. void main() {  
2.     var mapa = {'id': 1, 'name': 'Dart'};  
3.     print(mapa);  
4. }
```

Figura 2.10: Sintaxis de mapas en Dart.

6. Dinámico. Si el tipo de variable no está definido se declara dinámico.

```
1. void main() {  
2.     dynamic name = "Dart";  
3.     print(name);  
4. }
```

Figura 2.11: Sintaxis de datos dinámicos en Dart.

- **Ciclos y condicionales:** Admite las condicionales *if* ó *if else* similar al lenguaje Java, al igual el que tipo de condicional de cambio *switch*, así como los ciclos *for*, *for.in*, *while* y *do.. while*.

```
1. void main() {  
2.     dynamic name = "Dart";  
3.     if(name == "Dart") {  
4.         print("Si");  
5.     } else {  
6.         print("No");  
7.     }  
8. }
```

Figura 2.12: Sintaxis de condicional *if else* en Dart.

```
1. void main() {  
2.     dynamic n = 1;  
3.     switch(status) {  
4.         case 0:  
5.             print "No es el número";  
6.             break;  
7.         case 1:  
8.             print "Es el número";  
9.             break;  
10.        default:  
11.            print "No es el número";  
12.        }  
13. }
```

Figura 2.13: Sintaxis de condicional *switch* en Dart.

```
1. void main() {  
2.     for( var i = 1 ; i <= 10; i++ ) {  
3.         if(i%2==0) {  
4.             print(i);  
5.         }  
6.     }  
7. }
```

Figura 2.14: Sintaxis de ciclo *for* en Dart.

- **Clases y objetos:** Admite funciones de programación orientada a objetos como clases, interfaces, etc. Incluye lo siguiente:

1. Campos.
2. *Getters* y *setters*.
3. Constructores.
4. Funciones.

```
1.  class Persona {
2.      String nombre;
3.      String get nombre {
4.          return name;
5.      }
6.      void set nombre(String nombre) {
7.          this.nombre = nombre;
8.      }
9.      void nombreToString() {
10.         print(nombre);
11.     }
12. }
13. void main() {
14.     Persona persona = new Persona();
15.     persona.nombre = "Jhon";
16.     persona.nombreToString();
17. }
```

Figura 2.15: Sintaxis de clases y objetos en Dart.

Flutter pone la interfaz de usuario directamente en código Dart, usando la sintaxis de expresión normal de Dart:

```
1  class MyApp extends StatelessWidget {
2      @override
3      Widget build(BuildContext context) {
4          return MaterialApp(
5              title: 'Welcome to Flutter',
6              home: Scaffold(
7                  appBar: AppBar(title: Text('Welcome to Flutter')),
8                  body: Column(
9                      children: [
10                         Text('Hello World'),
11                         Text('This is Flutter'),
12                         Text('Written in Dart')
13                     ]
14                 )
15             );
16     });
17 }
18 }
```

Figura 2.16: Sintaxis de IU en Dart.

Todo después de esa palabra clave *return* es una expresión anidada que produce un fragmento de interfaz de usuario.

2.2.2. Características principales de Dart

1. Orientación a objetos: Todos los elementos del lenguaje son objetos, desde los tipos primitivos hasta las clases personalizadas, facilitando la modularización del código y su reutilización.
2. Tipado estático y dinámico: Se puede definir el tipo de dato de las variables en el momento de la declaración, pero también pueden optar por tipado dinámico cuando se requiere mayor flexibilidad.
3. Compilación JIT (*Just-In-Time*) y AOT (*Ahead-Of-Time*):
 - JIT. Permite la ejecución inmediata del código sin la necesidad de compilarlo completamente cada vez que se realiza un cambio. Esto es especialmente útil durante la fase de desarrollo, ya que habilita el “Hot Reload”, que permite ver los cambios en la interfaz de usuario de manera instantánea.
 - AOT. Se utiliza para crear versiones optimizadas del código que se ejecutan de manera eficiente en dispositivos móviles o de escritorio, mejorando la velocidad de ejecución de las aplicaciones.
4. Sintaxis familiar: Presenta una sintaxis sencilla y familiar con lenguajes como Java, JavaScript o C#, facilitando la curva de aprendizaje.
5. Multiplataforma: Puede ser utilizado para crear aplicaciones web y de escritorio, ofreciendo un único lenguaje que permite mantener una única base de código, simplificando el desarrollo y mantenimiento de proyectos complejos.
6. *Garbage collection*: Gestiona de manera eficiente la memoria a través de un proceso de recolección de basura (*garbage collection*), lo que permite liberar memoria de manera automática sin intervención del desarrollador, optimizando el uso de los recursos y mejorando el rendimiento general de las aplicaciones.

2.3. Visual Studio Code: Entorno de desarrollo

Visual Studio Code (también llamado VS Code) es un editor de código fuente desarrollado por Microsoft para Windows, Linux, macOS y Web. Incluye soporte para la depuración, control integrado de Git, resaltado de sintaxis, finalización inteligente de código, fragmentos y refactorización de código. Es un software gratuito y de código abierto, ofrece:

- Un rendimiento fluido incluso en computadoras con recursos limitados, ligero y rápido, ideal para proyectos que demandan eficiencia.

- La posibilidad de utilizar “Hot Reload”, una función que permite ver los cambios realizados en el código en tiempo real, sin necesidad de recompilar toda la aplicación.
- Integración con GitHub y plantillas de código para ayudar a compilar funciones de aplicaciones comunes y también importar código de muestra.
- Una interfaz intuitiva y opciones de personalización. Se pueden emplear extensiones como “Prettier” que mantiene el código limpio y formateado de manera consistente, además también se pueden incluir temas visuales para mejorar la organización y visualización de archivos.
- Extensiones que permiten la creación, depuración y ejecución eficiente de aplicaciones Flutter, que proveen un soporte completo para el lenguaje Dart y funcionalidades específicas para trabajar con *widets* y diseño responsivo.

Ofrece integración total con Flutter, lo que lo convierte en una herramienta ideal para crear aplicaciones multiplataforma desde un único entorno. Esto permite verificar el comportamiento de la aplicación en diferentes configuraciones de hardware y software, asegurando que la aplicación ofrezca una experiencia de usuario óptima en una amplia gama de dispositivos [11].

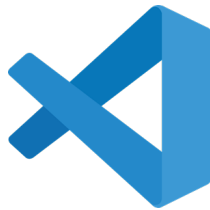


Figura 2.17: Logo de Visual Studio Code.

2.3.1. Git y GitHub para el control de versiones

Git junto con GitHub ofrece un control de versiones y almacenamiento de código. Juntos permiten y se diferencian en que:

Git	GitHub
Es un software.	Es un servicio.
Es instalado en el sistema localmente.	Esta alojado en una página Web.
Herramienta de línea de comandos.	Proporciona una interfaz gráfica.
Gestiona diferentes versiones realizadas en archivos en un repositorio Git.	Espacio para subir una copia del repositorio Git.
Proporciona funcionalidades como “Gestión del código fuente del sistema de control de versiones”.	Proporciona funcionalidades como “Administración de código fuente”, además de agregar características propias.

Tabla 2.3: Características de Git y GitHub.



Figura 2.18: Logo de Git y GitHub.

2.4. Arquitectura de aplicaciones móviles

La arquitectura de aplicaciones móviles se refiere a la estructura fundamental que define como se organiza y gestiona el código en una aplicación, así como la interacción de los componentes que la conforman [12]. Durante el desarrollo es esencial definir una arquitectura clara y eficiente que facilite el desarrollo, mantenimiento y escalabilidad. Los componentes clave de la arquitectura son:

- **Capa de presentación.** Es la capa con la que el usuario interactúa directamente. Incluye la interfaz de usuario (UI) y la experiencia de usuario (UX). En Flutter, esta capa se compone de *widgets*, la forma en la que la capa está organizada puede influir directamente en la fluidez y eficiencia de la aplicación.
- **Capa de servicios de negocios.** Es la parte del sistema que gestiona las reglas y la funcionalidad de la aplicación. Contiene la lógica de control sobre como los datos se procesan y como se realizan las operaciones solicitadas por el usuario. En Flutter, se pueden usar patrones como BloC (*Business Logic Component*) o

Provider para gestionar esta capa, manteniendo el código bien organizado y fácil de mantener.

- **Capa de datos.** Responsable de la gestión de los datos, ya sea que provengan de una base de datos local, archivos o fuentes externas, como APIs. La interacción entre la lógica de negocio y los datos debe ser fluida y optimizada para garantizar que la información llegue a la capa de presentación.

Los patrones de arquitectura más usados son:

- **MVC (*Model-View-Controller*).** Divide la aplicación en tres componentes principales:
 1. **Modelo:** Gestiona los datos y la lógica de la aplicación.
 2. **Vista:** Representa la interfaz de usuario.
 3. **Controlador:** Actúa como intermediario entre el modelo y vista, procesando entradas y actualizando la interfaz según sea necesario.

En Flutter este patrón es menos común, ya que puede generar dependencias entre la interfaz y la lógica de negocio, sin embargo es útil para aplicaciones pequeñas y sencillas.

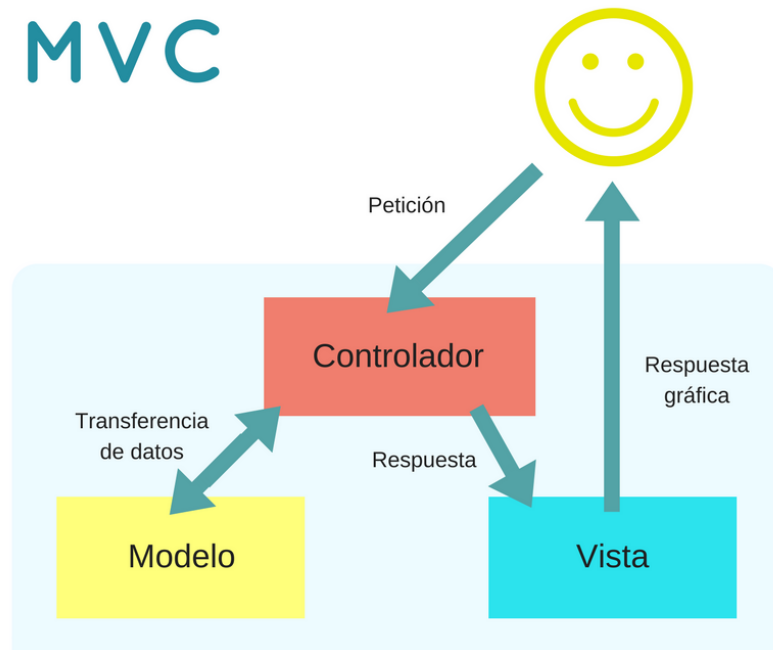


Figura 2.19: Diagrama del patrón MVC.

- **MVVM (*Model-View-ViewModel*).** Separa la lógica de presentación del resto de la aplicación:

1. Modelo: Representa los datos y la lógica de negocio.
2. Vista: Representa la interfaz de usuario.
3. *ViewModel*: Sirve como intermediario entre el modelo y la vista, manipulando los datos que la vista debe mostrar y manteniéndolos acoplados.

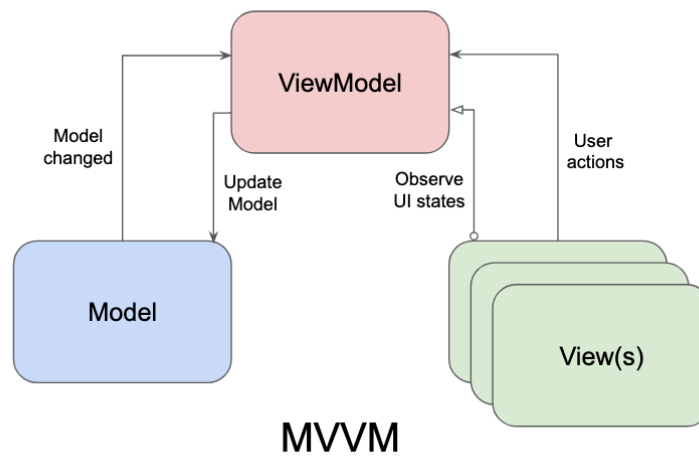


Figura 2.20: Diagrama del patrón MVVM.

- BloC (*Bussiness Logic Component*). Separa la lógica de negocio de la interfaz de usuario utilizando *Streams* para manejar eventos asíncronos, como la entrada del usuario o el procesamiento de datos:
 1. BloC: Gestiona la lógica de negocio y utiliza flujos de datos (*Streams*) para comunicarse con la interfaz de usuario.
 2. UI: La interfaz se suscribe a cambios emitidos por el BloC y actualiza los *widgets* según los nuevos estados recibidos.



Figura 2.21: Diagrama del patrón BloC.

2.5. Bases de datos y Firebase

Firebase es una plataforma móvil diseñada y creada por Google, teniendo como principal función desarrollar y facilitar la creación de aplicaciones para dispositivos móviles [13]. Se encuentra alojada en la nube y, por ende, está disponible para diferentes plataformas como Android, iOS, y web. Así mismo, cuenta con diversas funciones para que cualquier desarrollador pueda combinar y adaptar la plataforma a medida de sus necesidades.



Figura 2.22: Logo de Firebase.

Sus herramientas son variadas y de fácil uso, considerando que su agrupación simplifica las tareas de gestión a una misma plataforma. Las finalidades de las mismas se pueden dividir en cuatro grupos: desarrollo, crecimiento, monetización y análisis [13].

1. **Desarrollo:** Incluye los servicios necesarios para el desarrollo de un proyecto de aplicación móvil o web.
 - *Realtime database.* Se alojan en la nube, son *No SQL* y almacenan los datos como *JSON*. Permiten alojar y disponer de los datos e información de la aplicación en tiempo real, manteniéndolos actualizados aunque el usuario no realice ninguna acción. Aunque no hubiera conexión por parte de un usuario, sus datos estarían disponibles para el resto y los cambios realizados se sincronizarían una vez restablecida la conexión.
 - Autenticación de usuarios. Permite tanto el registro propiamente dicho (mediante email y contraseña) como el acceso utilizando perfiles de otras plataformas externas (por ejemplo, de Facebook, Google o X).
 - Almacenamiento en la nube. Permite guardar los ficheros de las aplicaciones (y vinculándolos con referencias a un árbol de ficheros para mejorar el rendimiento de la aplicación) y sincronizarlos.
 - *Crash reporting.* Detecta y ayuda a solucionar los problemas de la aplicación consiguiendo un informe muy detallado y organizado, agrupándolos por similitud y los clasifica por gravedad.
 - *Test lab.* Permite testear en dispositivos Android virtuales basados en parámetros configurados.
 - *Remote config.* Sirve para modificar ciertas funciones, aspectos o la apariencia de la aplicación sin que sea necesario publicar una actualización de la

misma. Considera factores como la ubicación o idioma del usuario, su dispositivo de acceso, etc.

- *Cloud messaging*. Envía notificaciones y mensajes a diversos usuarios en tiempo real y a través de varias plataformas.
- *Hosting*. Servidor para alojar las aplicaciones de manera rápida y sencilla, esto es, un hosting estático y seguro. Proporciona certificados de seguridad SSL y HTTP2 de forma automática y gratuita para cada dominio.

2. **Crecimiento:** Contempla tanto la gestión de aquellos que ya son usuarios de la misma, como herramientas para la captación de nuevas audiencias.

- *Notificaciones*. Con esta herramienta, se pueden diseñar y enviar las notificaciones push en el momento preciso, con la posibilidad de segmentarlas y personalizarlas (por ejemplo, en base al usuario, su idioma o el tipo de dispositivo que utiliza).
- *App indexing*. Posibilita la integración de la aplicación en los resultados arrojados por el buscador de Google. De este modo, las búsquedas sobre contenido relacionado pueden mostrar la aplicación indexada como resultado.
- *Dynamic links*. Permiten redirigir al usuario a zonas o contenidos concretos de la aplicación en función del objetivo que se quiera conseguir y de la personalización que se otorgue a diversos parámetros de esta URL.
- *Invites*. Los usuarios tienen la posibilidad de invitar a sus contactos a utilizar la aplicación o de compartir contenidos específicos con ellos.

3. **Monetización:** En este caso, la búsqueda de ganancias viene ligada a la publicidad que se puede insertar en las aplicaciones, Firebase cuenta con AdMob para rentabilizar la aplicación.

4. **Analítica:** El análisis de datos y resultados es clave para la toma de decisiones coherentes y fundamentadas para el proyecto. Con Firebase Analytics, se pueden controlar diversos parámetros y obtener mediciones variadas desde un mismo panel.

Capítulo 3

Marco metodológico

En el siguiente marco metodológico nos enfocaremos en el diseño e implementación así como el enfoque a seguir para el desarrollo de la aplicación móvil en Flutter para la automatización digital de reportes de incidentes para bomberos.

3.1. Enfoque metodológico

Este proyecto adopta un enfoque metodológico mixto, esto quiere decir que se integraran elementos tanto cualitativos como cuantitativos para garantizar un desarrollo completo y que se adapte a las necesidades reales del usuario final. Este enfoque consistirá en:

1. **Análisis cualitativo:** Comprender a profundidad las necesidades y expectativas del personal de los bomberos. Esto incluye el estudio de sus interacciones actuales con los reportes de incidentes y la identificación de las áreas de mejora.
2. **Análisis cuantitativo:** Evaluar el impacto de la implementación de la aplicación en términos de eficiencia, medido a través de indicadores como el tiempo de llenado de los reportes, cantidad de errores reducidos, así como el nivel de satisfacción del usuario final.

Para el desarrollo del proyecto se optará por la metodología ágil, la cual es un concepto de gestión de proyectos que implica dividir el proyecto en fases y hace hincapié en la colaboración y la mejora continuas, siguiendo un ciclo de planificación, ejecución y evaluación[14]. Esto nos permitirá realizar ajustes continuos en función de los comentarios de los usuarios finales asegurando que el producto final cumpla con los requerimientos.

La metodología ágil divide el proyecto en *sprints*, lo que asegura la entrega de avances funcionales de la aplicación en periodos cortos de tiempo. Además, esto fomentará la comunicación constante entre las partes involucradas del proyecto (miembros del equipo y usuarios finales) para alinear objetivos y solucionar problemas rápidamente.

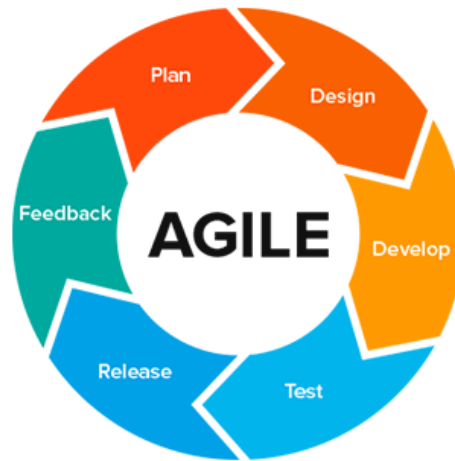


Figura 3.1: Diagrama de metodología ágil.

3.2. Requerimientos del usuario

Se realizó una entrevista con el personal de bomberos encargado del llenado de reportes para comprender cómo es su práctica actual y las dificultades asociadas al llenado manual de estos mismos. Entre los hallazgos clave que se comentaron se destacan:

- Frecuencia y repetitividad: Cada salida a incendio genera un reporte individual, lo que puede traducirse en un alto volumen de reportes por jornada. Por ejemplo, 16 servicios equivalen a llenar 16 reportes manualmente.
- Uso de herramientas tradicionales: Actualmente, los reportes se llenan con pluma azul y se suben a Google Drive de forma manual.
- Tiempo de llenado: Dependiendo de la complejidad, un reporte puede tomar entre 10 y 40 minutos, más 10 minutos adicionales para subirlo a la nube.

El análisis del tiempo de llenado nos permitirá establecer métricas para medir la eficiencia del sistema automatizado en términos de reducción de tiempo y esfuerzo. De igual forma, en el apéndice A se encontrará el formato oficial del llenado de los reportes.

Con todo lo anterior mencionado, la aplicación propuesta se diseñará para optimizar el flujo de trabajo actual mediante las siguientes funcionalidades:

- Digitalización del llenado de reportes: Reducirá significativamente los tiempos de elaboración gracias a formularios predefinidos y campos automáticos.
- Almacenamiento en Firebase: Los reportes serán almacenados directamente en una base de datos centralizada, desde donde podrán ser consultados, editados e impresos en cualquier momento.
- Generación de reportes en PDF: Listos para su impresión e almacenamiento según la necesidad del personal.
- Adaptación a dispositivos móviles: Garantiza accesibilidad desde cualquier dispositivo compatible.

El sistema se validará en colaboración con el personal de bomberos, realizando pruebas piloto para comparar los tiempos y la eficacia entre el método tradicional y el automatizado, recogiendo retroalimentación para potenciales mejoras.

3.3. Desarrollo de la aplicación

El desarrollo de la aplicación se realizó utilizando el lenguaje de programación Dart. En Dart, todo es un objeto y todas las clases heredan de la clase *Object*, lo que simplifica la estructura y potencia la reutilización de código. Se utilizó Visual Studio Code como entorno de desarrollo integrado (IDE) debido a su ligereza, expansibilidad y compatibilidad con Flutter. En esta sección se detalla el proceso llevado a cabo para el desarrollo inicial de la aplicación móvil de automatización de reportes de incidentes para bomberos. Se incluye el diagrama de funcionamiento de la aplicación, y la documentación preliminar de la estructura de archivos y código implementado.

3.3.1. Diseño de la interfaz

1. Diagrama de funcionamiento: A continuación se presenta un diagrama que representa el flujo de funcionamiento de la aplicación, destacando los procesos de llenado almacenamiento y consulta de reportes.

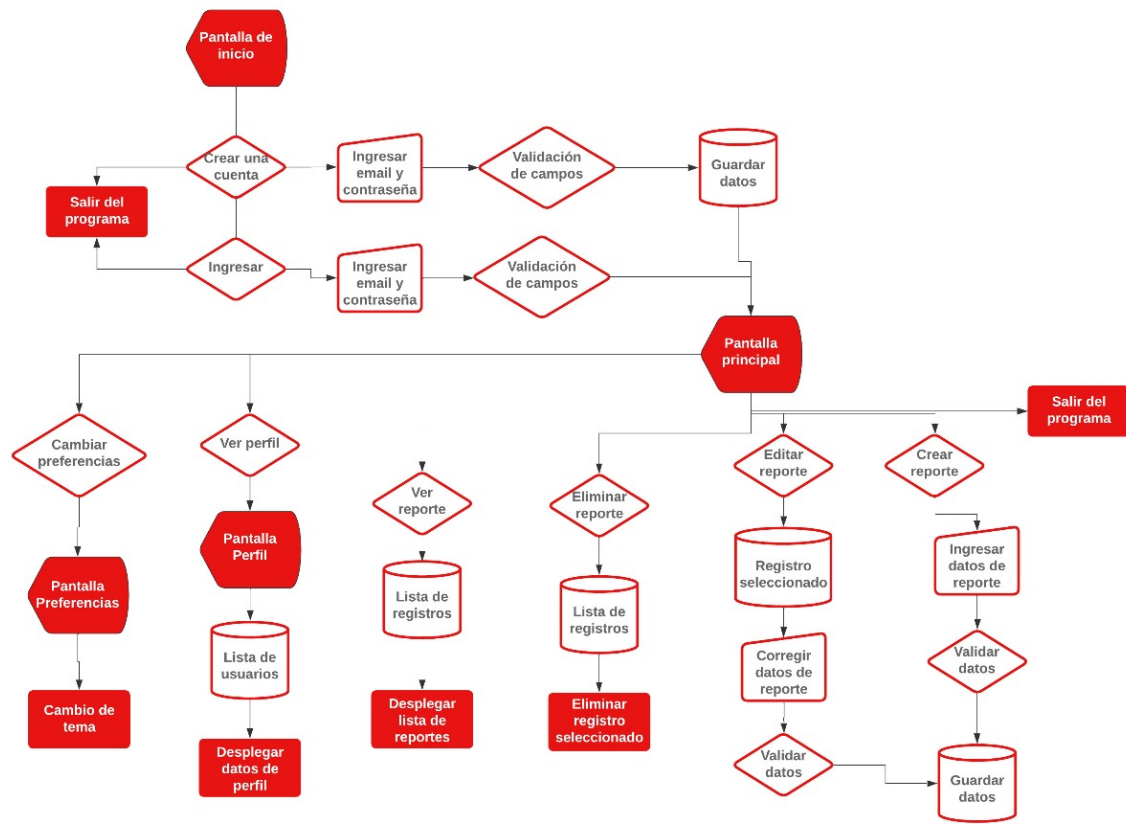


Figura 3.2: Diagrama de funcionamiento de la aplicación.

- Definición de la paleta de colores: Se trabajará con las herramientas “Coolors” junto con “Material Theme Builder”. Con la primer herramienta, se definió la siguiente paleta de colores como se observa en la figura 3.3.

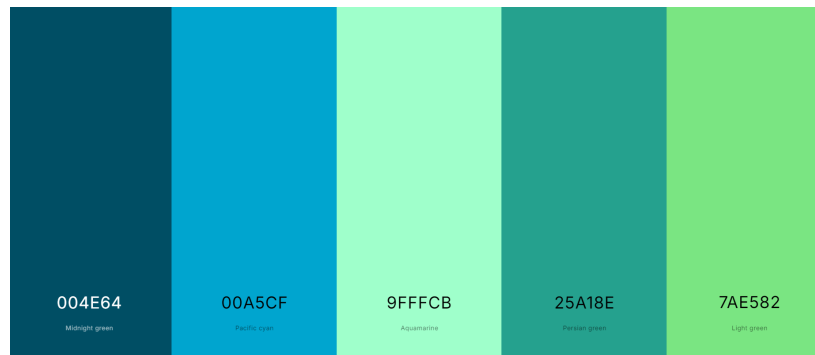


Figura 3.3: Paleta de colores seleccionado para la aplicación.

Una vez con esta paleta, usaremos “Material Theme Builder” que proporciona una vista previa de como se verán los colores dentro de la aplicación, además de generar directamente el archivo con el código necesario para implementar los colores.

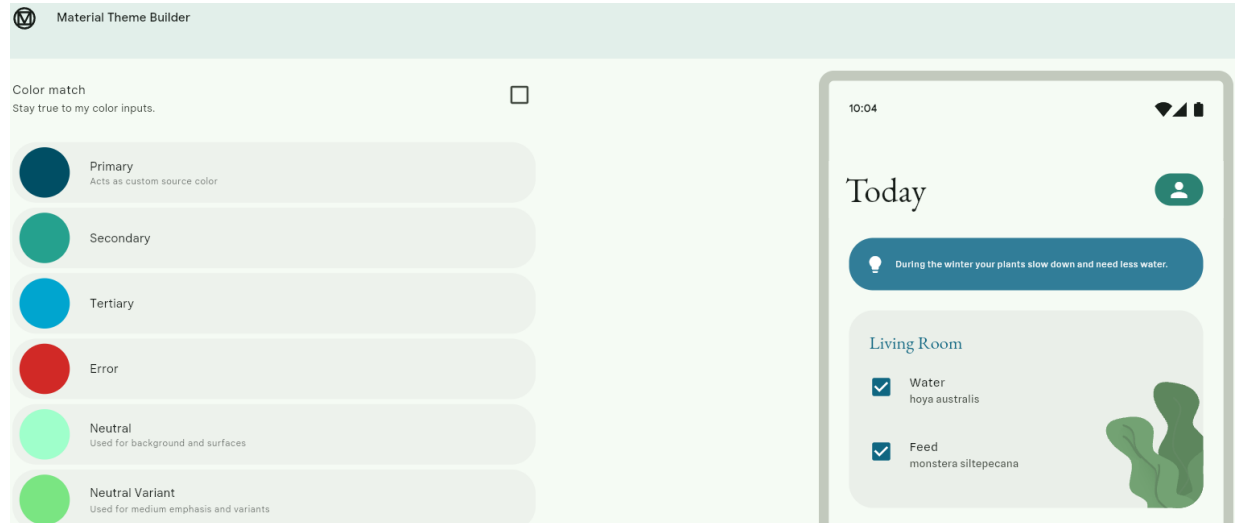


Figura 3.4: Colores finales seleccionados para la aplicación.

3.3.2. Estructura general de archivos y directorios

Flutter divide su proyecto en carpetas y archivos clave que facilitan el desarrollo, las carpetas principales son:

- *android* y *ios*: Contienen configuraciones específicas para cada plataforma.
- *assets*: Almacena imágenes y fuentes personalizadas.
- *lib*: Aquí se desarrolla el código principal de la aplicación.
- *components*: contiene principalmente *widgets* genéricos con el fin de mandarlos llamar cada vez que sea necesario a lo largo del desarrollo de otras pantallas.
- *utils*: Similar a *components*, es un directorio que tiene clases de uso genérico o de configuración global para la aplicación.
- *screens*: Este directorio contiene todas y cada una de las pantallas de la aplicación.
- *test*: Diseñada para pruebas unitarias.

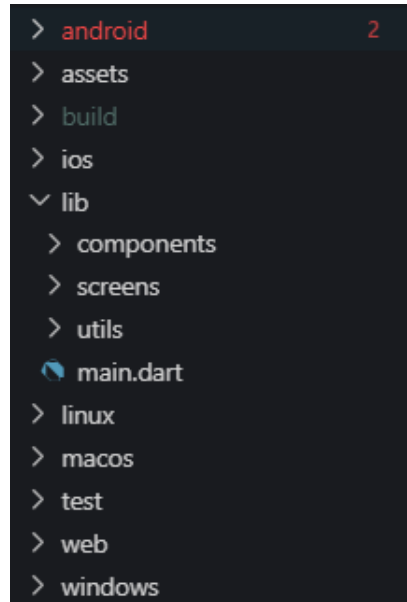


Figura 3.5: Carpetas principales del proyecto.

Main

El archivo *main.dart* contiene la configuración general de la aplicación.

```
void main() {  
  runApp(const MyApp());  
}  
  
class MyApp extends StatelessWidget {  
  const MyApp({super.key});  
  
  @override  
  Widget build(BuildContext context) {  
    return MaterialApp(  
      title: 'App Bomberos',  
      home: const LoginScreen(),  
      theme: const MaterialTheme(TextTheme()).light(),  
      routes: {  
        "/login": (context) => const LoginScreen(),  
        "/home": (context) => const DashboardScreen(),  
      },  
      debugShowCheckedModeBanner: false,  
    ); // MaterialApp  
  }  
}
```

Figura 3.6: Código de *main.dart*.

En este estado de la aplicación *main* contiene la siguiente configuración:

- Nombre de la aplicación.
- Pantalla por defecto al inicializar.
- Tema de colores y fuente.
- Rutas definidas para la navegación dentro de la aplicación.

Screens

- `login_screen.dart`. Su propósito es visualizar un formulario de inicio de sesión hacia la pantalla principal (*dashboard*). Todos los *widgets* de esta pantalla están alineados y centrados dentro de una columna, se muestra un logo y un mensaje de bienvenida así como también se mandan a llamar *widgets* dentro del directorio de *components* para renderizar y manejar información de campos de texto.

```
Image.asset(
  "assets/fireman_hat.png",
  width: 200,
), // Image.asset
const SizedBox(height: 50),
const Text('¡Bienvenido!',
  style: TextStyle(
    fontFamily: 'NexaDemo',
    fontWeight: FontWeight.w700,
    color: Colors.white,
    fontSize: 32,
  )), // TextStyle // Text
const SizedBox(height: 25),
MyTextField(
  controller: txtUserController,
  obscureText: false,
  hintText: 'Usuario',
), // MyTextField
const SizedBox(height: 10),
MyTextField(
  controller: txtPWDController,
  hintText: 'Contraseña',
  obscureText: true,
), // MyTextField
```

Figura 3.7: Código muestra del ícono y mensaje de bienvenida.

- `dashboard_screen.dart`. Dentro de esta pantalla se renderizarán más eficientizando los recursos para evitar que nuevas pantallas incluyan su propio *dashboard*. Contiene los siguientes elementos:
 - *Modal Drawer*: Menú de desplazamiento horizontal el cual muestra opciones adicionales, como cerrar sesión, desplegar información de la cuenta, foto de perfil, etc. Dichas opciones son conocidos como *ListTile* y permiten añadir ítems dentro de una lista con un formato atractivo visualmente.

```
Widget myDrawer() {
  return Drawer(
    child: ListView(
      children: [
        UserAccountsDrawerHeader(
          currentAccountPicture: ClipRect(
            borderRadius: BorderRadius.circular(100),
            //child: selectedImage != null ? Image.file(selectedImage!) : const Image(
            child: const Image(image: AssetImage("assets/pfp.jpg")), // ClipRect
            accountName: const Text("sun&moon"),
            accountEmail: const Text("zenwol@gmail.com"),
          ), // UserAccountsDrawerHeader
        ListTile(
          onTap: () {
            //Navigator.pushNamed(context, "/movies");
          },
          title: const Text("Perfil"),
          //subtitle: const Text("Lorem ipsum"),
          leading: const Icon(Icons.person),
          trailing: const Icon(Icons.arrow_forward_ios_sharp),
        ), // ListTile
      ],
    ),
  );
}
```

Figura 3.8: Código muestra de *ListTile*.

- *AppBar*: Contiene la opción de abrir el menú desplegable (*drawer*), el siguiente código implementa configuración del *AppBar* que hace cambiar su aspecto como el cambio de color, su título y una acción asociada al hacer “click” sobre el *IconButton*.

```
appBar: AppBar(
  backgroundColor: defaultColorScheme.secondary,
  leading: IconButton(
    onPressed: () {
      _scaffoldKey.currentState?.openDrawer(); // Abre el drawer
    },
    icon: Icon(
      Icons.menu,
      color: defaultColorScheme.onPrimary,
    ), // Icon
  ), // IconButton
  title: Text(
    "Reportes de incendios",
    style: TextStyle(
      fontFamily: 'NexaDemo',
      fontWeight: FontWeight.bold,
      color: defaultColorScheme.onPrimary), // TextStyle
  ), // Text
),
```

Figura 3.9: Código muestra de *AppBar*.

- *Floating Action Button*: Es un botón flotante localizable en la esquina inferior derecha de la pantalla, este botón tiene la utilidad de generar nuevos reportes.

```
floatingActionButton: FloatingActionButton(
  backgroundColor: defaultColorScheme.primary,
  onPressed: () {},
  child: const Icon(Icons.add, color: Colors.white, size: 28),
), // FloatingActionButton
```

Figura 3.10: Código muestra de *Floating Action Button*.

Components

- `my_button.dart`. Renderiza un botón donde sea que sea llamado, es un *widget* de uso general para toda la aplicación. Su diseño se basa principalmente en la decoración de un contenedor para dar el aspecto visual de un botón, esto permite una mayor personalización del componente.

```
@override
Widget build(BuildContext context) {
  final ColorScheme defaultColorScheme = Theme.of(context).colorScheme;
  return GestureDetector(
    onTap: onTap,
    child: Container(
      padding: const EdgeInsets.all(value: 25),
      margin: const EdgeInsets.symmetric(horizontal: 25),
      decoration: BoxDecoration(
        //color: const Color(0x91FF4444),
        color: defaultColorScheme.primary,
        borderRadius: BorderRadius.circular(radius: 8),
      ), // BoxDecoration
      child: const Center(
        child: Text(
          data: 'Iniciar sesión',
          style: TextStyle(
            fontFamily: 'NexaTextDemo',
            fontWeight: FontWeight.w700,
            fontSize: 17,
            color: Colors.white,
          ), // TextStyle
        ),
      ),
    ),
  ), // GestureDetector
```

Figura 3.11: Código muestra de `my_button.dart`.

- `my_textfield.dart`. Renderiza un campo de texto sea donde sea que lo llamen, para ello recibe tres parámetros en su constructor:
 - *Controlador*: El controlador permite extraer la información escrita en el campo de texto y manipularlo.
 - *hint*: Texto que se despliega a forma de guía o ayuda para saber que escribir por parte del usuario.

- *obscureText*: Cambia el formato de visualización por uno que no sea visible.

```
const MyTextField(
  {super.key,
   required this.controller,
   required this.hintText,
   required this.obscureText});

@override
Widget build(BuildContext context) {
  return Padding(
    padding: const EdgeInsets.symmetric(horizontal: 25.0),
    child: TextField(
      controller: controller,
      obscureText: obscureText,
      decoration: InputDecoration(
        enabledBorder: const OutlineInputBorder(
          borderSide: BorderSide(color: Colors.grey)), // OutlineInputBorder
        focusedBorder: const OutlineInputBorder(
          borderSide: BorderSide(color: Color(0xff6262ff)), // OutlineInputBorder
          fillColor: Colors.grey[250],
          filled: true,
          hintText: hintText,
          hintStyle: TextStyle(color: Colors.grey[500]), // InputDecoration

```

Figura 3.12: Código muestra de *my_textfield.dart*.

Utils

Guarda una serie de temas basados en los siguientes colores que ofrece “Material Theme”: *primary*, *secondary*, *tertiary*, *neutral*, *error* y *surface*. Se basa en la normas de diseño y configuración UI/UX dadas por Google.

```
const MaterialTheme(this.textTheme);

static ColorScheme lightScheme() {
  return const ColorScheme(
    brightness: Brightness.light,
    primary: Color(0xff0f6681),
    surfaceTint: Color(0xff0f6681),
    onPrimary: Color(0xffffffff),
    primaryContainer: Color(0xffbce9ff),
    onPrimaryContainer: Color(0xff001f29),
    secondary: Color(0xff026b5d),

```

Figura 3.13: Código muestra de *theme.dart*.

3.4. Técnicas de validación

En esta etapa inicial del proyecto, la validación solo se enfocará en verificar los aspectos visuales y estructurales de la aplicación, con vistas a validar su funcionalidad completa en el futuro.

3.5. Documentación

No se generará una documentación formal ni una guía de usuario, ya que el desarrollo se encuentra enfocado solamente en la creación del prototipo visual y la estructura básica de la aplicación.

Capítulo 4

Pruebas y resultados

A continuación se mostraran las tres pantallas principales de la aplicación desarrolladas hasta el momento como parte del prototipo en el cual se irá trabajando para adecuarse a las necesidades para el llenado de reportes de incidentes.

Se evaluó la consistencia visual de las pantallas desarrolladas, verificando:

1. Colores, tipografía y alineación de elementos.
2. Diseño atractivo e interfaz intuitiva.

Para esto se utilizó la inspección manual en Visual Studio Code con la ejecución en emuladores. Además se realizó la validación de la navegación entre las pantallas principales:

- Pantalla de *Login*.
- Pantalla de selección de reportes y *dashboard*.

Iniciando la aplicación se encuentra la pantalla de *Login*:



proyecto_integrador_bomberos



¡Bienvenido!

Usuario

Contraseña

[¿Olvidó su contraseña?](#)

Iniciar sesión

[¿No está registrado? Regístrese ahora](#)

Figura 4.1: Pantalla de *Login* de la aplicación.

Como se observa, se cuenta con los campos de llenado para el usuario así como su respectiva contraseña, además de un botón para iniciar sesión junto con la posibilidad de recuperar contraseña o registrar un nuevo usuario.

Ingresando tenemos la vista donde se podrán visualizar, editar, agregar o eliminar reportes:

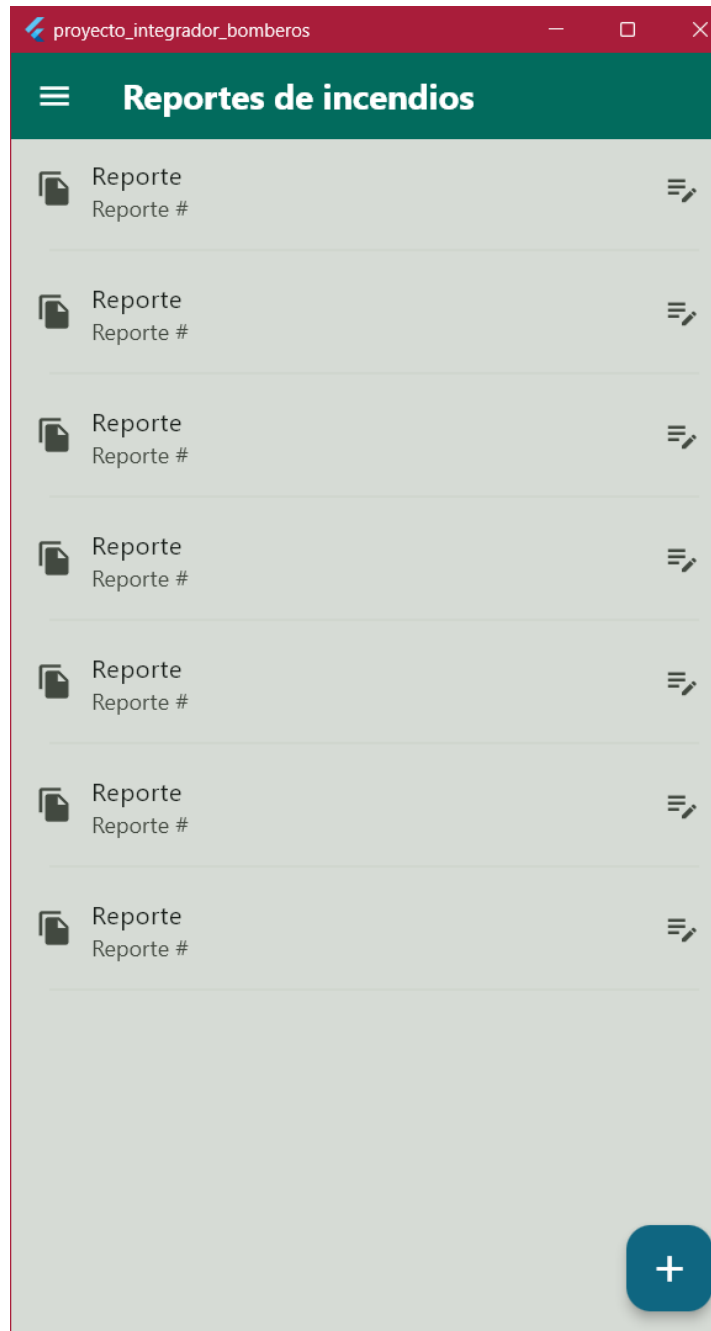


Figura 4.2: Pantalla de reportes de la aplicación.

Y finalmente si tocamos el ícono que se encuentra en la esquina superior izquierda, nos muestra un *dashboard* con más posibles acciones a realizar:

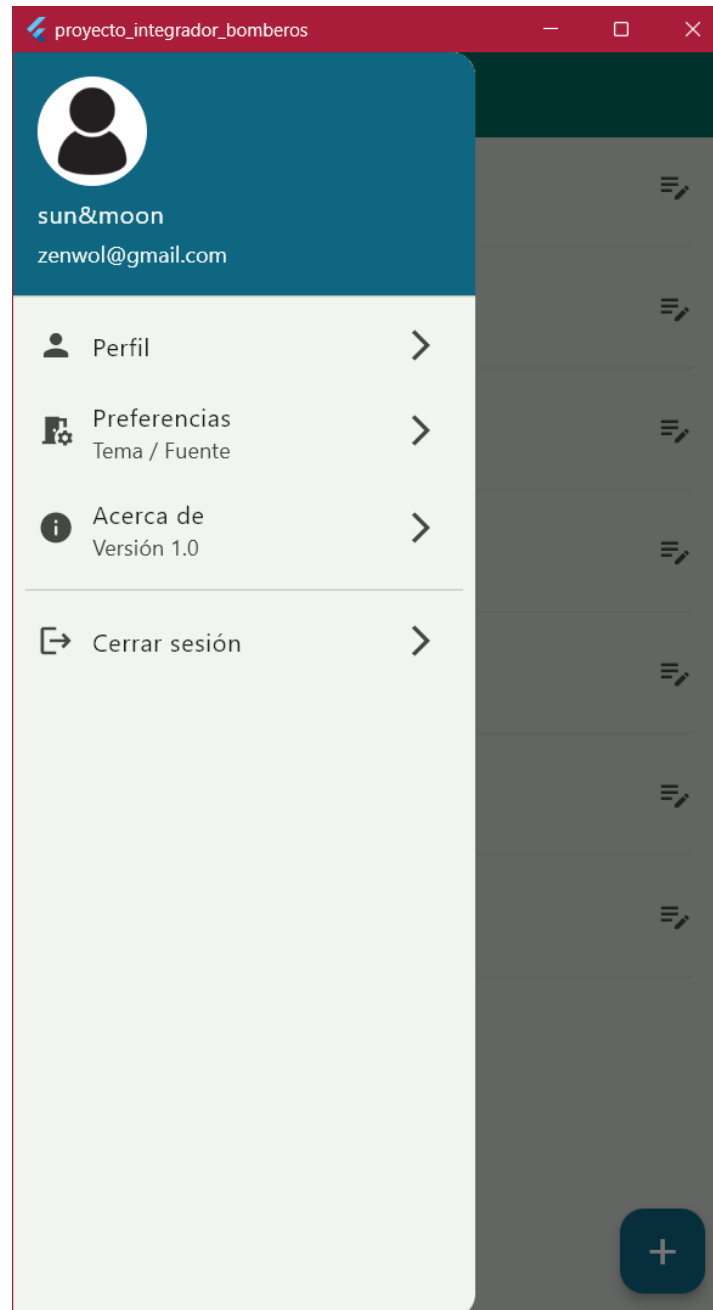


Figura 4.3: Pantalla del *dashboard* de la aplicación.

Como se puede observar en la figura 4.3, se tienen diferentes opciones a realizar, en donde el usuario podrá cambiar su foto de perfil, acceder a este, cambiar el tema de la aplicación, conocer más acerca de la aplicación y su versión o cerrar sesión para ingresar con otro usuario.

Capítulo 5

Conclusiones

El proyecto de la aplicación para la automatización de reportes de incidentes para bomberos representa para nosotros como equipo un verdadero esfuerzo significativo, debido a que ninguno de los integrantes antes había trabajado con la programación móvil y mucho menos en un lenguaje como Dart o con el *framework* de Flutter, aún así creemos que esta aplicación modernizará y hará más eficiente los procesos de documentación de emergencias. Durante la etapa actual, se ha logrado establecer una base sólida para el desarrollo inicial de la interfaz de usuario y la planificación del sistema.

Como equipo, hemos tenido que ser autodidactas y repasar constantemente el como funciona el entorno de Flutter con Visual Studio Code, a pesar de las limitaciones actuales en la aplicación, como la falta de validación en el inicio de sesión o la funcionalidad completa de los formularios, se ha avanzado en la conceptualización del flujo de trabajo y la visualización de la aplicación. Esto nos proporcionó un marco claro para poder continuar con el desarrollo.

Con todo esto, como equipo y con los resultados obtenidos hasta el momento, ha sido satisfactorio la realización del prototipo de la aplicación ya que nos dio una base teórica y nos hizo pensar más allá del solo verlo como una implementación a cumplir, sino que nos brindó más conocimientos con respecto a las buenas prácticas de trabajo en equipo así como dentro de la programación.

5.1. Trabajo a futuro

A medida que el proyecto avance, se trabajará en la integración y validación de las siguientes funcionalidades principales:

- Desarrollo de un sistema de inicio de sesión funcional con ayuda de Firebase, para registrar usuarios así como sus contraseñas.

- Implementación de formularios interactivos que permitan registrar todos los datos necesarios, con validaciones para evitar errores de registro.
- Integración de Firebase para guardar los reportes en tiempo real, asegurando la accesibilidad de los datos.
- En una etapa más avanzada del proyecto, se generará una documentación detallada desde una guía de usuario, donde se explique paso a paso el uso de todas las funcionalidades de la aplicación, hasta la documentación técnica donde se incluirá una descripción detallada de la arquitectura del sistema.

Además, en etapas posteriores, se implementarán técnicas para asegurar la calidad del sistema, incluyendo:

- Pruebas unitarias. Validación de cada módulo o funcionalidad desarrollada para garantizar que operen correctamente de manera individual.
- Pruebas de integración. Verificación de la interacción entre todos los módulos de la aplicación, asegurando el flujo de datos.
- Pruebas de usabilidad. Revisión del diseño de la interfaz con los usuarios finales para identificar mejoras.
- Pruebas de rendimiento. Evaluar la capacidad del sistema para manejar grandes volúmenes de datos.

Este trabajo a futuro asegurará cumplir con los objetivos iniciales planteados, ofreciendo un sistema robusto y eficiente.

Bibliografía

- [1] F. C. W. Andreina and V. S. F. Santiago, “Aplicacion movil para la automatizacion del sistema de atencion a emergencias 9-1-1.” 2023. [Online]. Available: <https://repositorio.unphu.edu.do/handle/123456789/5615>
- [2] D. Corral, “Propuesta de app movil para la gestion de incidentes de transito - proquest.” [Online]. Available: <https://www.proquest.com/openview/dc587bdc2c3025c679850d69a704dcf7/1?pq-origsite=gscholar&cbl=1006393>
- [3] Trabajo de Investigacion y/o Extension, UNIVERSIDAD DE CORDOBA, 2020. [Online]. Available: <https://repositorio.unicordoba.edu.co/server/api/core/bitstreams/d94199e1-c035-4631-a866-08af479d36d4/content>
- [4] R. R. J. Gabriel, “Desarrollo de un sistema web para el control de registro de emergencias en las guardias bomberiles del cuerpo de bomberos del canton la libertad,” Jun. 2022. [Online]. Available: <https://repositorio.upse.edu.ec/handle/46000/7707>
- [5] “Desarrollo de una aplicación android para la comunicación entre equipos de bomberos,” 2019. [Online]. Available: <https://riuma.uma.es/xmlui/bitstream/handle/10630/19199/Puga%20Roldan%20Ruben%20Memoria.pdf?sequence=1&isAllowed=y>
- [6] S. Chaves, “Que es flutter como funciona,” Aug. 2023. [Online]. Available: <https://formadoresit.es/que-es-flutter-como-funciona/>
- [7] Tiagofur, “Descubriendo el mundo de los widgets en flutter: Construye tu interfaz de usuario perfecta.” Oct. 2024. [Online]. Available: <https://creapolis.dev/descubriendo-el-mundo-de-los-widgets-en-flutter-construye-tu-interfaz-de-usuario-perfecta/>
- [8] [Online]. Available: <https://docs.flutter.dev/resources/architectural-overview>
- [9] J. G. Gomila, “Que es dart historia, características y ventajas de aprenderlo,” Oct. 2024. [Online]. Available: <https://cursos.frogamesformacion.com/pages/blog/que-es-dart>
- [10] sacavix.com, “Flutter: Generalidades del lenguaje dart - sacavix tech,” Oct. 2020. [Online]. Available: <https://sacavix.com/2020/10/flutter-generalidades-del-lenguaje-dart/>
- [11] J. Hernandez, “Visual studio code para el desarrollo de flutter,” *Medium*, May 2022. [Online]. Available: <https://devjaime.medium.com/visual-studio-code-para-el-desarrollo-de-flutter-256dbd13b453>

- [12] “Descripcion de capas logicas (descripcion general tecnica de sun java enterprise system 5).” [Online]. Available: <https://docs.oracle.com/cd/E19528-01/820-0888/aaubbb/index.html#:~:text=La%20capa%20de%20servicios%20de%20negocio%20consiste%20en%20la%20%C3%B3gica,de%20datos%20o%20sistemas%20heredados>.
- [13] S. L. Mora, “Firebase: que es, para que sirve, funcionalidades y ventajas,” Oct. 2022. [Online]. Available: <https://digital55.com/blog/que-es-firebase-funcionalidades-ventajas-conclusiones/>
- [14] Atlassian, “Que es agil?” [Online]. Available: <https://www.atlassian.com/es/agile>

Apéndice A

Reporte de incidentes de bomberos

En este capítulo se incluirá el formato del reporte de incidentes que utilizan actualmente el cuerpo de bomberos.

Información Complementaria	
Nombre del Afectado: _____	
Función que Desempeña: _____	
Dirección: _____	
Teléfono: _____	

Material Involucrado y Daños Ocasionados.	

Personal en Servicio.	
Operador: _____	Jefe de Servicio: _____
Asistente: _____	Asistente: _____
Asistente: _____	Asistente: _____
Asistente: _____	Asistente: _____

Unidades de Bomberos J.R. de Apoyo		
Unidad: _____	Encargado: _____	No. Elementos: _____
Unidad: _____	Encargado: _____	No. Elementos: _____
Unidad: _____	Encargado: _____	No. Elementos: _____
Unidad: _____	Encargado: _____	No. Elementos: _____

Unidades de Otras Instituciones			
Institución: _____	Unidad: _____	Encargado: _____	No. Elementos: _____
Institución: _____	Unidad: _____	Encargado: _____	No. Elementos: _____
Institución: _____	Unidad: _____	Encargado: _____	No. Elementos: _____
Institución: _____	Unidad: _____	Encargado: _____	No. Elementos: _____
Institución: _____	Unidad: _____	Encargado: _____	No. Elementos: _____
Institución: _____	Unidad: _____	Encargado: _____	No. Elementos: _____

Observaciones:

Firma de Jefe de Guardia: _____